A Microservices-Based Home Energy Tracker for Event-Driven Monitoring, Deduplicated Alerting, and ML-Based Anomaly Detection

Rohit Shrivastava, Anshika Rai, Harsh Gupta, and Syamasudha Veeragandham

School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India

rohit.lalbahadur2023@vitstudent.ac.in; anshika.rai2023@vitstudent.ac.in; harsh.gupta2023@vitstudent.ac.in; syamasudha.v@vit.ac.in

Abstract - This paper reports an evidence-bounded evaluation of a microservices-based Home Energy Tracker that couples Wokwi-compatible telemetry, gateway controls, Kafka-mediated processing, time-series usage storage, deterministic insights, Random Forest inference, and alert deduplication. The legacy supervised baseline was reproduced offline from a copied 5,000-record CSV using a chronological 70/15/15 split. The reproduced Random Forest achieved precision 0.880952, recall 0.925000, F1 0.902439, ROC-AUC 0.997430, and PR-AUC 0.948532 on the 750-record held-out interval; a 300-draw stratified bootstrap gave a conditional 95% F1 interval of 0.838300-0.957444. Five input-contract cases and 19 isolated component tests passed. An in-process, single-worker inference microbenchmark measured 98.719 ms mean and 129.680 ms p95 latency across 300 predictions. The study also identifies a Wokwi/training energy-scale mismatch, so simulator outputs are not treated as field evidence. Earlier synthetic end-to-end and local load records are retained only as historical observations. The results support deterministic reproduction and selected component behavior, not physical-meter accuracy, external household generalization, production capacity, or comprehensive fault tolerance.

Index Terms - home energy monitoring, microservices, event-driven architecture, anomaly detection, Wokwi, reproducibility, evidence classification.

I. INTRODUCTION

Residential energy applications often combine sensing, storage, analytics, and notification, but a working prototype does not by itself establish whether an anomaly model is reproducible or whether an event-driven pipeline has been evaluated beyond a demonstration. This work examines a Home Energy Tracker as an engineering system and makes the boundary between implemented mechanisms and measured evidence explicit. The system receives Wokwi-compatible telemetry, routes it through an API gateway, distributes events through Kafka, stores usage, computes deterministic energy insights, applies a persisted classifier, and handles threshold and anomaly alerts.

The study focuses on traceability rather than a claim of a new anomaly-detection algorithm. It retains the existing microservices architecture and evaluates the available supervised baseline on a copied local CSV. To prevent conflation of evidence types, results are classified as previously recorded historical evidence, newly reproduced evidence, newly generated isolated evidence, or not tested/blocked. The current Docker environment was not started, restarted, queried, or modified for these experiments because its recovery remains separately gated.

A. Objectives and Contributions

The objectives are to preserve a clear telemetry-to-alert architecture, reproduce the documented supervised baseline without changing the deployed system, evaluate selected input and component behaviors in an isolated local path, and communicate the resulting evidence with bounded claims.

- An event-driven implementation that separates gateway control, ingestion, event distribution, usage persistence, deterministic insights, ML inference, alert processing, and monitoring.
- A reproducible chronological evaluation of the supplied supervised Random Forest and Logistic Regression baseline, including data-quality checks, split-overlap guards, fixed-model bootstrap uncertainty, and artifact comparison.
- A current evidence package containing input-contract checks, an explicit simulator/training-scale finding, isolated component-test results, and an in-process inference microbenchmark, while preserving historical integration/load observations as historical.

II. RELATED WORK AND RESEARCH GAP

A. Existing Approaches

Recent work has examined microservice and cloud architectures for residential or smart-grid energy management [1], [2], [4], [5], while smart-meter studies demonstrate the value of anomaly detection over consumption data [1], [3]. These directions motivate a system that preserves the distinction between telemetry transport, energy insight functions, and learned anomaly scoring. The official REFIT dataset is a suitable future source of documented aggregate and appliance power data, but it was not used to derive any quantitative result in this paper [6].

B. Research Gap and Scope

The identified gap is not merely the absence of another monitoring dashboard. It is the need for a traceable evaluation that connects a microservices energy-monitoring architecture with reproducible model evidence and clearly bounded operational claims. The present system addresses this gap by reproducing its available baseline and separating fresh isolated tests from earlier runtime records. It does not close the broader research gap of external household validation because the supplied CSV lacks documented source, licence, label provenance, and causal provenance for Avg_Past_Consumption.

III. SYSTEM ARCHITECTURE AND IMPLEMENTATION

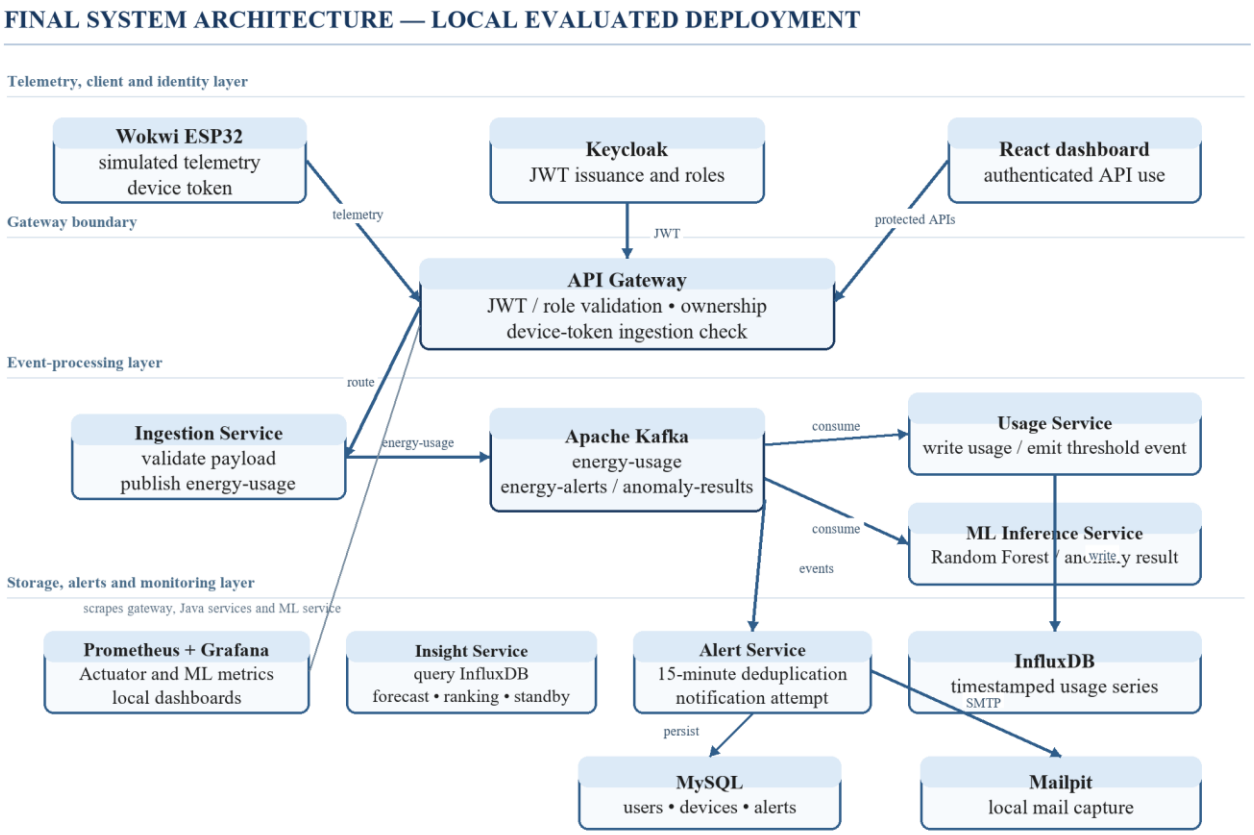


Fig. 1. Implemented Home Energy Tracker architecture. Solid arrows denote request or event flow. The diagram describes the architecture; it does not imply that every link was re-executed in the current offline experiment.

A. Event-Driven Telemetry Flow

The telemetry contract contains deviceId, energyConsumed, and timestamp. User-facing routes are designed to use Keycloak-issued JWTs, whereas simulator-compatible ingestion uses a device-bound route. Gateway, ingestion, usage, ML, and alert services have distinct responsibilities: payload validation and acceptance, energy-usage event publication, time-series persistence and threshold handling, anomaly scoring, and alert processing. The current manuscript distinguishes this static architectural path from the parts for which fresh execution evidence exists.

Kafka mediates the documented energy-usage, energy-alert, and anomaly-result flows. Usage records are intended for InfluxDB, while user, device, and alert metadata are intended for MySQL. The React dashboard and Mailpit mail sink are presentation/capture components. These design roles are retained from the code and prior local evidence; current work did not invoke these services or access their persistent stores.

B. Deterministic Insights and Alert Handling

For a D-day period and aggregate energy E, the implementation derives daily consumption E/D and a 30-day projection 30E/D. Its tariff, ranking, and standby calculations are deterministic application functions, not learned forecasts. The alert service forms a user/device-or-household and alert-type key before applying its configured 15-minute suppression window. Historical local records exist for threshold alerts and duplicate suppression, but no new alert or notification event was generated for this revision.

C. Supervised ML Pipeline and Runtime Contract

The available supervised CSV has seven fields: Timestamp, Electricity_Consumed, Temperature, Humidity, Wind_Speed, Avg_Past_Consumption, and Anomaly_Label. The trainer chronologically sorts input, maps Normal and Abnormal to a binary target, and derives Hour, DayOfWeek, and ConsumptionDeviation = Electricity_Consumed - Avg_Past_Consumption. The target is excluded from the feature contract. A Random Forest with 300 trees and a Logistic Regression baseline use the same chronological split; the decision threshold is selected on validation F1 rather than on held-out test data.

Runtime inference may receive only deviceId, energyConsumed, and timestamp. When optional contextual fields are absent, the ML code reports their imputation using stored medians. The current input-contract experiment exercised this code path in-process without starting an ASGI server, sending an HTTP request, or opening a Kafka connection. It therefore confirms local contract behavior, not deployed inference behavior.

The new evidence also reveals a deployment-relevance issue: the training Electricity_Consumed range is 0-1, while documented Wokwi scenarios use 1-6, 8-20, and 26-40 kWh bands. This scale difference must be resolved by unit and feature alignment before simulator-driven predictions can be interpreted as meaningful energy-anomaly evidence.

D. Evidence Classification

Evidence is classified as A (previously recorded historical), B (newly reproduced), C (newly generated isolated), or D (not tested/blocked). The reproduced baseline uses a copied CSV and isolated output artefacts. Bootstrap, input-contract, component, and microbenchmark results are C-class. Earlier end-to-end, load, deduplication, and outage records remain A-class because they were inspected but not rerun. Full current deployment tests are D-class because Docker recovery and REFIT deployment remain separately unauthorized.

This classification avoids presenting a mock component test as an end-to-end test or a simulator input as a physical-meter measurement. It also permits the architecture to be described without misrepresenting its availability or operational status during the evidence collection.

E. Operational Workflow

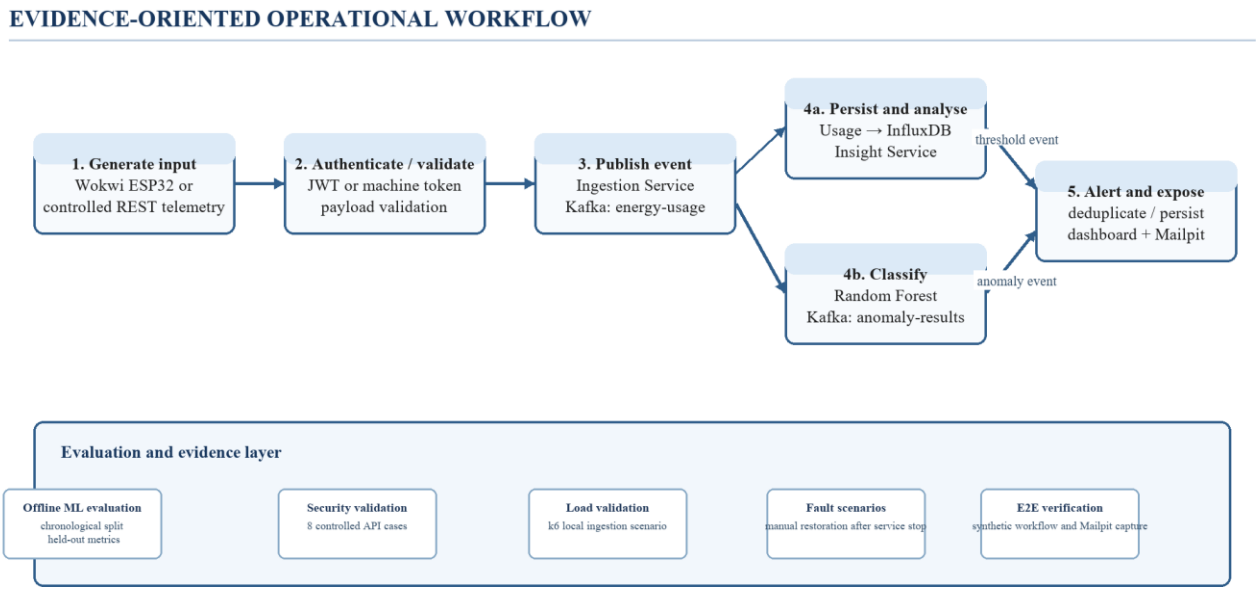


Fig. 2. Architecture workflow and evidence boundaries. The upper path depicts the implemented system; the lower band is interpreted as evidence categories rather than a claim that all runtime paths were newly exercised.

Algorithm 1 summarises the intended processing path. Lines that publish events, persist usage, or deliver notifications are implementation steps. Fresh results in this paper are limited to the offline and component-level portions explicitly identified in Sections IV and V.

Algorithm 1. Secure telemetry-to-alert event processing

```
Input: telemetry t = {deviceId, energyConsumed, timestamp}; caller credential c.
```

```
Output: accepted-event status; downstream artifacts after asynchronous consumers
complete.
1: Validate required fields, timestamp, and non-negative energyConsumed.
2: Verify an eligible caller credential for the intended route.
3: Route accepted telemetry through the gateway to ingestion.
4: Publish the energy-usage event for asynchronous consumers.
5: Persist usage and evaluate deterministic threshold conditions.
6: Build the available ML feature vector, record any contextual imputation, and
publish an anomaly result.
7: Apply the alert deduplication key and window before persistence and local
notification attempt.
```

IV. EXPERIMENTAL DESIGN AND IMPLEMENTATION

A. Implemented Components and Evidence Boundaries

TABLE I. Implemented components and evidence boundaries

Layer	Implemented component(s)	Role and evidence boundary
Input	Wokwi ESP32 simulator; controlled REST client	Wokwi-compatible synthetic telemetry; not a calibrated energy meter.
Identity and gateway	Keycloak; API Gateway	JWT/ownership/device-token route design; current evidence includes only offline gateway-filter tests.
Event processing	Ingestion, Usage, ML Inference, Alert services; Kafka	Event publication, persistence, inference, threshold/anomaly handling; no current live-service run.
Data	InfluxDB; MySQL	Intended usage series and metadata stores; not accessed in current experiments.
Presentation and notification	React frontend; Mailpit	Dashboard and local mail capture; previous synthetic record retained as historical evidence.
Monitoring	Prometheus; Grafana; Actuator / ML metrics	Architecture component; current metrics/health collection was not rerun.

TABLE II. Execution environment and evidence classes

Item	Recorded configuration	Evidence class and scope
Host operating system	Windows 11, build 26300; 8 logical CPUs	B/C offline experiments run on this host.
Python ML environment	Python 3.13.2; pandas 2.3.3; NumPy 2.5.3; scikit-learn 1.9.1	B/C copied-data reproduction and in-process testing.
Dataset input	Copied smart_meter_data.csv; SHA-256 d48df940...f588	B; source/label provenance remains unresolved.
Application integration	Prior local Docker Compose synthetic record	A historical evidence only; not restarted or queried.
Load and fault checks	Prior k6 and controlled service-stop records	A historical observations; no new live workload or fault injection.

B. Dataset and Model Configuration

TABLE III. Supplied CSV characteristics and chronological split

Characteristic	Verified value
Dataset identity	smart_meter_data.csv; repository does not document source, licence, physical unit, or label-generation provenance.
Raw shape	5,000 rows; Timestamp, five numeric predictors, and Anomaly_Label.
Time coverage	2024-01-01 00:00:00 to 2024-04-14 03:30:00; 30-minute interval.
Data checks	0 invalid timestamps/numeric values, 0 negative energy values, and 0 complete duplicate rows in the copied CSV.
Class distribution	4,750 Normal (95.0%); 250 Abnormal (5.0%).

Characteristic	Verified value
Chronological split	Train 3,500 (3,335/165); validation 750 (705/45); test 750 (710/40), Normal/Abnormal.
Feature contract	Five raw numeric fields plus Hour, DayOfWeek, and ConsumptionDeviation; target excluded.

TABLE IV. Candidate-model configuration

Model	Configuration	Threshold selection
Logistic Regression	StandardScaler; LogisticRegression(max_iter=1000, class_weight=balanced, random_state=42)	Validation F1; 0.05-0.95 search in 0.01 increments.
Random Forest	300 trees; min_samples_leaf=2; class_weight=balanced_subsample; random_state=42	Validation F1; selected threshold 0.31.

C. Evaluation Protocol

1. Offline baseline reproduction: copy the supplied CSV, verify its SHA-256, reapply the stable chronological split and feature transformation, select the threshold on validation F1, and report untouched held-out metrics. This is B-class evidence.
2. Statistical and input checks: load the isolated reproduced model, hold the threshold fixed, use 300 stratified bootstrap draws on the test interval, and exercise validation/imputation cases in-process. Controlled Wokwi values are treated as synthetic interface inputs, not observations. These are C-class evidence.
3. Component checks: run only in-process or pure-local tests that do not invoke Docker, HTTP, a broker, database, persistent volume, credential, replay, or live service. The current set contains 19 passing tests; unavailable Kafka-worker modules are reported as blocked rather than passing. These are C-class evidence.
4. Performance: record a separate in-process single-worker inference microbenchmark. Do not combine this latency with endpoint, Kafka, or alert latency. The pre-existing k6 observation is reported only as A-class historical local evidence.
5. Historical integration: retain the dated synthetic end-to-end and local Mailpit observations with original provenance, but do not call them fresh validation. The existing stack was not restarted or used in this study.
6. Scientific boundaries: do not infer household generalization from the CSV because label provenance and causal availability of Avg_Past_Consumption remain unresolved; do not infer physical-meter accuracy from Wokwi values.

V. RESULTS AND DISCUSSION

A. Newly Reproduced Supervised Baseline

The copied CSV SHA-256 matched the historical dataset record. It contains 5,000 rows, 4,750 Normal and 250 Abnormal labels, and a stable chronological 3,500/750/750 split. No timestamp, complete raw-record, or engineered-feature fingerprint overlapped across split pairs. The reproduced Random Forest and Logistic Regression metrics matched the historical comparison exactly, with zero absolute difference for every reported metric. The Random Forest was selected using validation F1 = 0.914894 and achieved held-out precision 0.880952, recall 0.925000, F1 0.902439, ROC-AUC 0.997430, and PR-AUC 0.948532. A 300-draw stratified bootstrap on the fixed test interval yielded conditional 95% intervals of 0.787234-0.971840 for precision, 0.850000-1.000000 for recall, 0.838300-0.957444 for F1, 0.994489-0.999526 for ROC-AUC, and 0.891762-0.992244 for PR-AUC.

TABLE V. Newly reproduced held-out CSV metrics

Model	Threshold	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.28	0.05333	1.00000	0.10127	0.42285	0.05971
Random Forest	0.31	0.88095	0.92500	0.90244	0.99743	0.94853

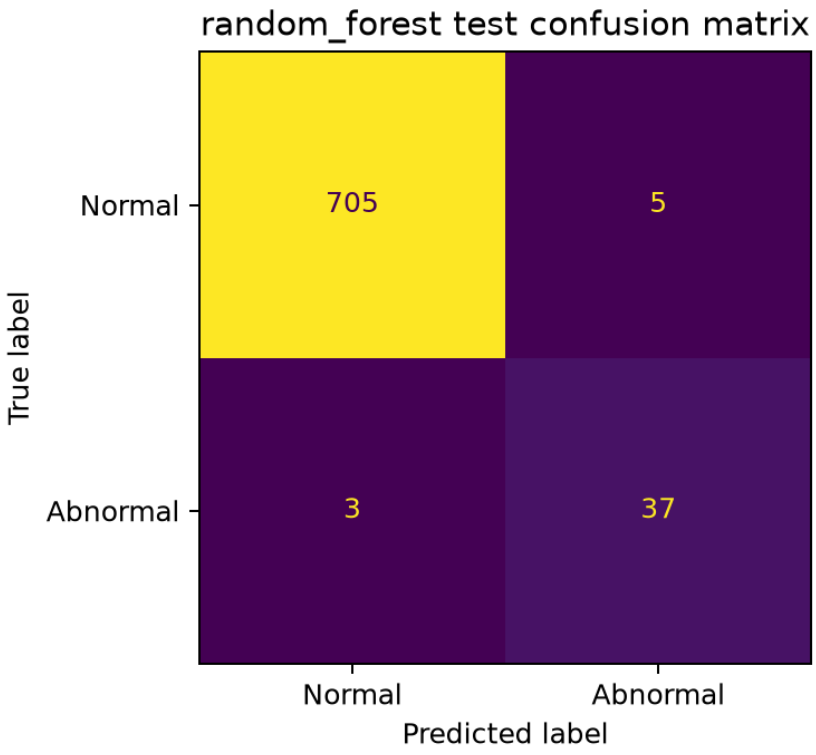


Fig. 3. Confusion matrix for the reproduced Random Forest on the 750-record held-out CSV interval: 705 true negatives, 5 false positives, 3 false negatives, and 37 true positives.

The metrics and confidence intervals are conditional on the supplied labelled CSV, the fixed chronological split, the trained model, and its validation-selected threshold. They do not resolve the undisclosed source and label-generation process, nor can row-level checks establish whether Avg_Past_Consumption was derived without access to future observations. These limitations prevent a claim of external household performance.

B. Newly Generated Input and Component Evidence

All five expected input-contract cases passed: zero energy was accepted; negative energy, a missing device identifier, and an unexpected field were rejected; and a naive timestamp was normalised to UTC. Missing Temperature, Humidity, Wind_Speed, and Avg_Past_Consumption were explicitly identified as imputed by the inference path. Of six documented Wokwi energy boundaries, five lay above the training Electricity_Consumed maximum of 1. The controlled inputs produced no ground-truth labels, so this is a scale-compatibility finding rather than an accuracy result. Nineteen of nineteen fresh selected component tests passed; they cover gateway filter logic, in-memory REFIT runtime/trainer logic, and replay-timestamp monotonicity. Two Kafka-worker test modules were not executed because the local interpreter lacked the Kafka dependency.

TABLE VI. Newly generated isolated input and component evidence

Current isolated evidence	Observed result	Interpretation and boundary
Input validation	5/5 expected outcomes passed.	In-process contract behavior; not gateway or deployment validation.
Wokwi scale check	5/6 documented boundaries above training electricity maximum.	Synthetic inputs without labels; compatibility finding, not accuracy.
Gateway filter tests	8/8 passed.	Mock reactive objects; no live issuer, endpoint, or token was used.
ML runtime/trainer tests	10/10 passed.	Pure in-memory unit tests; no REFIT replay or real data experiment.
Replay timestamp helper	1/1 passed.	Standard-library unit test; no telemetry transmitted.

C. Historical Integration and Security Evidence

The retained integration record is A-class evidence. In a prior single-host synthetic case, three submissions of 0.25, 0.50, and 0.75 kWh produced a recorded 1.50 kWh total and analytics became available after 3.79 s. A separate 1.00 kWh synthetic input above a 0.10 kWh threshold yielded a recorded threshold alert and local Mailpit capture after 13.64 s. These are dated observations with polling included; they were not rerun here. Likewise, the historical eight-case gateway HTTP suite is retained as historical evidence, while the current eight fresh gateway-security-filter tests are in-process mocks rather than a live security assessment.

TABLE VII. Evidence classification across system tests

Evidence item	Class and observed result	Interpretation
Baseline model reproduction	B: exact metric match to prior record.	Validates deterministic replay of the repository baseline on the copied CSV.
Bootstrap uncertainty	C: 300 stratified draws; F1 95% CI 0.838300-0.957444.	Conditional on fixed model, split, and supplied CSV.
Input contract	C: 5/5 expected cases passed.	Validates ML contract only; not live authorization.
Gateway security filters	C: 8/8 passed.	Mock component behavior; not a penetration test.
ML/trainer logic	C: 10/10 passed.	In-memory unit behavior; no server/broker execution.
Replay timestamp helper	C: 1/1 passed.	Local monotonicity helper only.
Synthetic integration	A: 1.50 kWh case; analytics 3.79 s; threshold alert 13.64 s.	Historical single-host observation, not rerun.
Local k6 scenario	A: 450/450 accepted; p95 109.77 ms.	Historical local run, not a capacity claim.

After a reactive gateway response-body correction, four focused regression tests passed and eight protected dashboard routes returned non-empty HTTP 200 JSON responses in the dated local runtime validation. Historical broader service tests (28 Java tests with 3 skipped) and 3 ML API tests are reported as earlier, separate runs; they are not combined into a falsely contemporaneous total.

D. Performance and Resilience Boundaries

The new in-process benchmark loaded the isolated reproduced model with one worker and evaluated 100 evenly spaced held-out rows three times (n = 300 predictions). Mean, median, p95, and p99 single-record latency were 98.719, 96.332, 129.680, and 152.312 ms, respectively; three 100-row batch runs averaged 586.344 rows/s. These values exclude HTTP, queueing, database, and alert-delivery work. The previously recorded k6 result and controlled service-stop observations are historical and remain separately reported in Table VIII. No new full-stack fault, queue-recovery, or notification delivery experiment was performed.

The final Prometheus snapshot reported eight active targets, each with health up, and local health checks returned HTTP 200 for the frontend, gateway health endpoint, Prometheus readiness endpoint, Grafana health endpoint, and Mailpit API. Cumulative suppression counters existed for ML_ANOMALY and THRESHOLD alert types. No derived alert rate, Kafka lag, distributed trace, or measured ML latency is reported because such evidence was not retained.

TABLE VIII. Performance and resilience evidence boundaries

TABLE VIII. Local load and controlled fault observations

Experiment	Observed result	Evidence-bound interpretation
New ML microbenchmark	300 single-record predictions; mean 98.719 ms; p95 129.680 ms; p99 152.312 ms; 586.344 rows/s mean batch throughput.	In-process, single-worker local model timing only; excludes HTTP, Kafka, database, and alert work.
Historical k6 ingestion	450/450 accepted; 0.00% HTTP failure; mean 39.44 ms; p95 109.77	A-class single-host historical run; not rerun and not a

Experiment	Observed result	Evidence-bound interpretation
	ms.	production-capacity result.
Historical service-stop cases	Gateway ingress accepted during recorded Mailpit and ML-service stops; manual recovery was recorded.	A-class observation only; no demonstrated replay, automatic recovery, or durable notification delivery.

The combined evidence supports deterministic reproduction of the repository baseline and selected offline component behavior. It does not demonstrate sustained service recovery, durable delivery, exactly-once processing, production security, or distributed scalability. In particular, Kafka capacity and Docker recovery remain outside the scope of this manuscript update.

E. Discussion and Threats to Validity

The most defensible contribution is a reproducible, evidence-aware systems study: the architecture integrates telemetry, access control, event processing, usage records, deterministic insights, ML inference, and deduplicated alert handling, while the associated evaluation labels what was actually run. Internal validity is strengthened by chronological splitting, duplicate and overlap checks, validation-only threshold selection, exact reproduction, and fixed-model bootstrap analysis. Construct validity remains constrained by undocumented CSV provenance and Wokwi-unit alignment. External validity is limited by the lack of a documented multi-household or field-data evaluation. The results should therefore be framed as a controlled prototype evaluation rather than a Q1-ready household-energy deployment study.

VI. REPRODUCIBILITY, LIMITATIONS, AND FUTURE WORK

All numerical claims in this revision are traceable to `experiments/publication_final/experiment_results.csv` and the associated evidence manifest. The copied dataset hash, Python package versions, model metadata, scripts, raw metric outputs, component-test summaries, and benchmark samples are retained in the repository. Reproduction uses isolated output paths and does not require application credentials or a running Docker stack. The current data limitations are material: the CSV source, licence, units, label-generation process, and causal availability of Avg_Past_Consumption are not independently documented.

Future work should first align the ingestion energy unit and contextual features with training data. A scientifically stronger evaluation would use a documented licensed household dataset such as REFIT [6], preserve household identity and temporal ordering, and evaluate household-held-out performance. Separate isolated deployment approval is needed before testing authorization, Kafka delivery, persistence, alert deduplication, notification capture, fault recovery, and load at runtime. Controlled perturbations must remain labelled as controlled rather than observed real anomalies.

VII. CONCLUSION

This paper presents an evidence-bounded Home Energy Tracker built around an event-driven microservices architecture. The available Random Forest baseline was reproduced exactly on the supplied chronological CSV and obtained held-out F1 0.902439 with a conditional 95% bootstrap interval of 0.838300-0.957444. Fresh isolated evidence also includes five input-contract checks, 19 component tests, and a single-worker inference microbenchmark. By contrast, integration, load, and fault observations are explicitly retained as historical records, and the Wokwi/training energy-scale mismatch is reported as an unresolved deployment-validity issue. The study therefore supports reproducibility and selected component behavior, while leaving real-household validation, full-stack resilience, and journal-level generalization as necessary future work.

REFERENCES

[1] A. Shuaibu, S. U. Haq, B. Almadani, F. Aliyu, and E. Al-Nahari, "Smart Metering Meets AI: Real-Time Appliance Monitoring and Anomaly Detection Over DDS Middleware," IEEE Transactions on Industry Applications, vol. 62, no. 2, pp. 2326-2342, 2026, doi: 10.1109/TIA.2025.3608687.

[2] I. Nunez, C. A. Rovetto, E. Cruz, A. Smolarz, D. H. Concepcion, and E. Cano, "Design of a microservices-based architecture for residential energy efficiency monitoring," International Journal of Electronics and Telecommunications, vol. 70, no. 4, pp. 1089-1098, 2024, doi: 10.24425/ijet.2024.152511.

[3] O. Hernandez et al., "Detection of Anomalies in Daily Activities Using Data from Smart Meters," Sensors, vol. 24, no. 2, art. no. 515, 2024, doi: 10.3390/s24020515.

[4] A. F. S. Veloso, J. Reis, and R. Rabelo, "A Microservices-Based Solution with Hybrid Communication for Energy Management in Smart Grid Environments," Sensors, vol. 26, no. 5, art. no. 1714, 2026, doi: 10.3390/s26051714.

[5] L. L. Motta, W. S. Alvarez, H. R. Carvajal, F. B. Neto, E. D. Lima, and L. F. P. Meloni, "A cloud architecture for home energy management systems: a conceptual model," Energy Informatics, vol. 8, art. no. 142, 2025, doi: 10.1186/s42162-025-00599-1.

[6] D. Murray, L. Stankovic, and V. Stankovic, "REFIT: Electrical Load Measurements," University of Strathclyde, 2015, doi: 10.15129/31da3ece-f902-4e95-a093-e0a9536983c4.